```
public static void main( String args[] )
{
double money = 0.0;
Employee empl;
empl = new RetiredEmployee();
money = empl.calc_pay();
System.out.println( "Money=" + money );
}
}
```

Caution, this works only when you're calling a method that exists in your Superclass, not one that exists only in the Subclass. And so now you see the advantage of identifying a bunch of empty methods in your Superclass. And the whole process is called, Polymorphism.

5.5 Polymorphism

We can create a Superclass reference that is an array. Then, when we instantiate the array, we can attach all different kinds of Subclass objects to the Superclass array reference. As long as we're only calling methods that exist in the Superclass, we can work our way through the array and it will call all the correct overridden versions of each individual Subclass object. The Superclass reference only knows about the methods that exist in the Superclass. The Superclass only tries to call the methods it knows about. That's perfectly fine, because all the methods in the Superclass are available in its Subclass.

Therefore the Superclass isn't aware that the object it references can do a whole lot more than the Superclass thinks it can. So, "We can create a Superclass reference array that actually points to a Subclass object." As long as we treat those Subclass objects as if they were Superclass objects, (meaning we only call the methods in the Superclass) we have no problems. The Subclass knows how to do everything its parent Superclass can do. The kids can do everything the parent can.

However, if we try to do it the other way around, treating the Superclass as if it were one of its children then we can have problems. The Subclass can do many things the Superclass cannot. The kids can do many things the parent cannot. If we called the Superclass with the Subclass's reference, then you might expect the Subclass reference can do all the things the kids can and you'd be wrong. If you want to go the other way, attach a Superclass object to a Subclass reference, you have to do an explicit cast, as a way of informing the